Cybersecurity Portfolio (Part 2)

Isaac Rutter (24273992)
https://github.com/saacutter/cybersecurity-pf/tree/main/Part%202

Table of Completion

Activity	Description	Completeted	Self-Assessment
B1	Discover 5 unique weak/vulnerable security implementations.	×	0
B2	Discover 5 unique strong security implementations.	×	0
B3	Discover 3 proactive security implementations in practice.	×	0
B4	Participate in 3 in-class activities in labs (facilitators will administer such activities).	✓	2
B5	Attend 2 cybersecurity related talks/seminars.	✓	2
B6	Attend 2 cyber ethics/law related talks/seminars.	×	0
B7	Participate in CS/DS/cybersecurity clubs' activity.	×	0
B8	Participate in a hackathon.	×	0
B9	Participate in an industry-related cybersecurity event.	×	0
B10	Participate in a cybersecurity activity as part of job/internship/volunteering etc.	×	0
B11	Talk to 2 cybersecurity experts from the industry and find out their latest projects.	×	0
B12	Discover two bias cases when using a generative AI system.	×	1
B13	Perform a jailbreak attack on a generative AI assistant (controlled test only).	×	0
B14	Teach your friends about cybersecurity topic of your choice.	×	0
B15	Teach an elderly person about cybersecurity topic of your choice.	✓	1
B16	Survey the current state-of-the-art solutions in cybersecurity.	×	0
B17	Implement one of the current state-of-the-art solutions and evaluate it.	×	0
B18	Contribute to an open-source project related to cybersecurity.	×	0
B19	Find and fix a vulnerability from a GitHub project.	×	0
B20	Enhance the security of a GitHub project.	×	0
B21	Design and implement a cybersecurity learning activity.	✓	2
B22	Enhance the cybersecurity of a website from your community.	×	0
B23	Test an intrusion detection system and discuss its effectiveness.	×	0
B24	Design and implement access control of your choice.	×	0
B25	Design and implement a threat intelligence module of your choice.	×	0
B26	Help another student in this unit struggling to understand/learn a cybersecurity concept.	×	0
B27	Apply a learned concept in this unit to a real-world application/problem/environment.	×	0
B28	Produce a cyber safety flyer for (choose 1): elders, high school students, CEOs, Uni students.	×	0
B29	Find a CVE in this year and fix it using three different generative AI systems (e.g., ChatGPT, Gemini), comparing the consistency.	✓	2
B30	Generate an AI-created image, applying an imperceptible watermark on it and then perform an image-to-image regeneration or editing process to make sure the watermark is detectable—the watermark survives.	×	0

B1. Discover 5 unique weak/vulnerable security implementations.

...

B2. Discover 5 unique strong security implementations.

...

B3. Discover 3 proactive security implementations in practice.

...

B4. Participate in 3 in-class activities in labs (facilitators will administer such activities).

To complete this activity, I attended 3 different labs - March 12, April 2, and April 23. These each covered different topics from the previous week's lecture, like encryption, security tools, and intrusion detection systems. For each activity, I have provided varied evidence which should prove completion.

Activity 1 - Thursday, March 12 (Week 3)
For this activity, everyone got into groups of 4-5. Each person came up with a random number (which for me was whatever number came to my head first, which was 148), and the goal was to calculate the average of every person's number without anyone finding out anyone else's chosen number.

Initially, we tried formatting the number in some way with some calculation (i.e. multiplying/dividing/adding/subtracting by an arbitrary amount) and writing it on a list as an "encrypted" number (for this, I just halved my number resulting in an "encrypted" number of 74). Then, each person could average the numbers written, using their actual numbers instead of their "encrypted" numbers once they got to that position in the list. Once everyone had an average, each person could say their average and then the average of the averages calculated. The thought process behind this would be that the "encrypted" numbers would be averaged out of the equation, with only each person's numbers left. This, however, did not work so we tried to come up with another solution.

The next attempt we instead decided to split our numbers up into 3 components (where all 3 add up to our numbers), write these on a piece of paper, and transfer it to the 3 people on our left (because by this point, the group's size had grown to 7 people). The totals each person had could then be written down, and the average of each person's sum could be calculated. This should have worked in theory, but with the amount of people in the group it did not end up working too well (although, according to the lab facilitator it was really close).

The group was then split into 2 to try and limit mistakes due to the amount of people. This same process was done, but instead of splitting the number up into 3 pieces it was instead split into 2. These pieces were then written down on pieces of paper, and a piece given to the two people on our lefts. Each person then calculated the sum of the numbers they were given, and the average of the sums was calculated. This finally gave us the correct average, with a result of 67.5 (the only number I am aware of in this sum was my number, but from this result it could very well have been the maximum).

Evidence for this activity was hard to get. Taking photos of the pieces of paper would not realistically prove anything, nor would taking a photo of me in the room. This write-up is intended to act as evidence, as it would be impossible to know what happened in the lab nor write about a process without having attended this lab.

Activity 2 - Thursday, April 2 (Week 6)
For this activity, everyone split into groups of 3 (though my group had 4 due to the number of people). The task was website vulnerability scanning, and then reporting our findings to the group. To do this, I used the Docker image of OWASP ZAP and scanned http://demo.testfire.net. I then used the results of this to discuss with the other group members.

The evidence provided is a photo taken by one of the group members, as well as a photo of me using OWSAP ZAP.

Activity 3 - Thursday, April 23 (Week 8)
This activity was focused on intrusion detection systems, specifically focusing on anomaly-based systems. Although this activity required everyone to split into groups of 4, the lab initially only had 3 people so we acted as a group of 3 (with the lab facilitator acting as the fourth). This activity used playing cards acting as network traffic, with one person acting as the network putting the deck of cards down in quick succession. The other people had to try and filter the traffic by looking at the card and grabbing it from the pile if it was marked "malicious" (with a reference provided by the facilitator) before the next one could be put down. When all the cards were put down, the cards which were grabbed from the pile were pooled to identify the number of true positives, true negatives, false negatives, and false positives.

A total of three attempts of this activity were made. None of these attempts were particularly successful, but some of the malicious "traffic" was pulled out with only a few false positives and false negatives.

The evidence for this activity is the initial outcome of the cards I pulled from the pile, as well as a photo of the malicious card reference list.

B5. Attend 2 cybersecurity related talks/seminars.

For this activity, I attended two online seminars focusing on business cybersecurity. Although the target audience for these seminars/talks were for businesses, not only would some of the topics be useful for completing the project but they covered some topics that I found interesting.

Cyber Wardens Foundations

Cyber Wardens is a government funded initiative (with support from Telstra, Commonwealth Bank, and the Council of Small Business Organisations of Australia) which aim to help small business owners become aware of cybersecurity threats that could impact their business. They offer several free services to accomplish this goal, including frequent seminars with guest speakers (including those from both public and private organisations). For this activity, I actually attended two separate seminars:

- A "foundations" seminar on March 18, which had a guest speaker from the Australian Signals Directorate (ASD)
- Another "foundations" seminar on March 19, which had a guest speaker from the Australian Federal Police (AFP)

The first seminar I attended was on March 18. This seminar had Rebecca Paterson from the Australian Signals Directorate as a guest speaker. The full notes of this seminar, some screenshots from the call, and the email confirmations are available in the `Cyber Wardens - ASD` directory. The seminar began with Rebecca discussing some of the impacts of cybercrime in the last year (including some statistics) and quickly spoke about how businesses (of all sizes) can prevent cyberattacks and what they can do if they become a victim to a cyberattack. This segment only lasted for roughly the first 15 minutes before moving onto the Cyber Wardens presentation. This presentation went a bit more in-depth about the same topics, focusing on identifying and preventing cyberattacks on small businesses and what they can do if they become a victim. This presentation ran for approximately 30 minutes, then moved onto the final Q&A session which filled up the remaining 15 minutes of the seminar. In this session, they just answered some of the questions that other attendants had posted in the chat during the presentation. There wasn't a lot of important information in this section, but that was to be expected. Overall, this seminar was completely fine. It wasn't much value to me personally which was expected, but to a small business owner there was some excellent information that could genuinely help them secure their business against cyberattacks.

The next seminar I attended was on March 19. This seminar had Claudia Forsyth, the Team Lead for Cyber Crime Prevention at the Australian Federal Police, as a guest speaker. The full notes of this seminar, some screenshots from the call, and the email confirmations are available in the `Cyber Wardens - AFP` directory. This seminar followed pretty much the exact same structure as yesterday. It began with Claudia discussing some of the impacts of cybercrime on Australia, the top cybercrime threats for businesses in Australia (with a specific focus on business email compromise), and how businesses can protect against and report cybercrime. This was much longer than the ASD's equivalent segment, running for the first 28 minutes. It then moved onto the Cyber Wardens presentation, which was essentially the exact same as yesterday's presentation. This finished at about 8:54, where they went on to answering questions from other people in attendance. Nothing of note was really answered in this session, which is unlike yesterday's session (possibly because the AFP's presentation was longer and explained things better but I don't know for sure). Overall, I would say this seminar was better than yesterday's. It still is targeted towards small business owners, but covering one topic in-depth rather than brushing on a few different topics probably makes it easier for business owners to digest (and business email compromise being a big attack vector right now makes the most sense to cover in-depth). Although there wasn't a lot of specific information on how to harden security, the prevention techniques presented would probably be adequate enough for most businesses.

NAB Business Cyber Security Awareness

This seminar was held by the National Australia Bank (NAB), one of the big four banks in Australia. This is one of a series of semi-frequent seminars hosted by NAB which provide business owners with an overview of the cybersecurity threat landscape of today. Although this one was primarily targeted towards businesses, they also offer some seminars talking about how cybercriminals are targeting individuals.

I attended the seminar hosted on April 7. This seminar was presented by Silvana Macri, a member of NAB Security Group situated at the Perth office. The full notes of this seminar, some screenshots of the call, and the email confirmations are available in the `NAB` directory.

This seminar began with some discussion on the current cyber threat landscape in Australia. It gave some details on what cybercriminals look like, how to respond to cyber threats, and moved onto the main topic of the seminar - business email compromise (BEC). This covered topics like signs of suspicious emails, the types of BEC, and how to respond to it if your organisation becomes a victim to it. Once this section was done, it moved onto expectations for how the cyber threat landscape will change in 2026 (and beyond) before moving onto what safeguards NAB has in place for these attacks. Finally, it moved onto a Q&A session at 10:48 before finishing around 10:54.

This seminar was similar the previous seminars which I had attended. It covered similar topics with similar pacing, but both had some information which would be highly relevant to those running businesses and not particularly "cyber aware". The only criticism I have with this seminar is that it didn't really feel like a NAB seminar, and could easily have been from any other organisation for the most part. But that could also be a positive, depending on how you look at it, as it means the information is accurate and easily digestable for small-medium business owners.

B7. Participate in CS/DS/cybersecurity clubs' activity.

...

B12. Discover two bias cases when using a generative AI system.

...

B13. Perform a jailbreak attack on a generative AI assistant (controlled test only).

...

B14. Teach your friends about cybersecurity topic of your choice.

...

B15. Teach an elderly person about cybersecurity topic of your choice.

...

B16. Survey the current state-of-the-art solutions in cybersecurity.

...

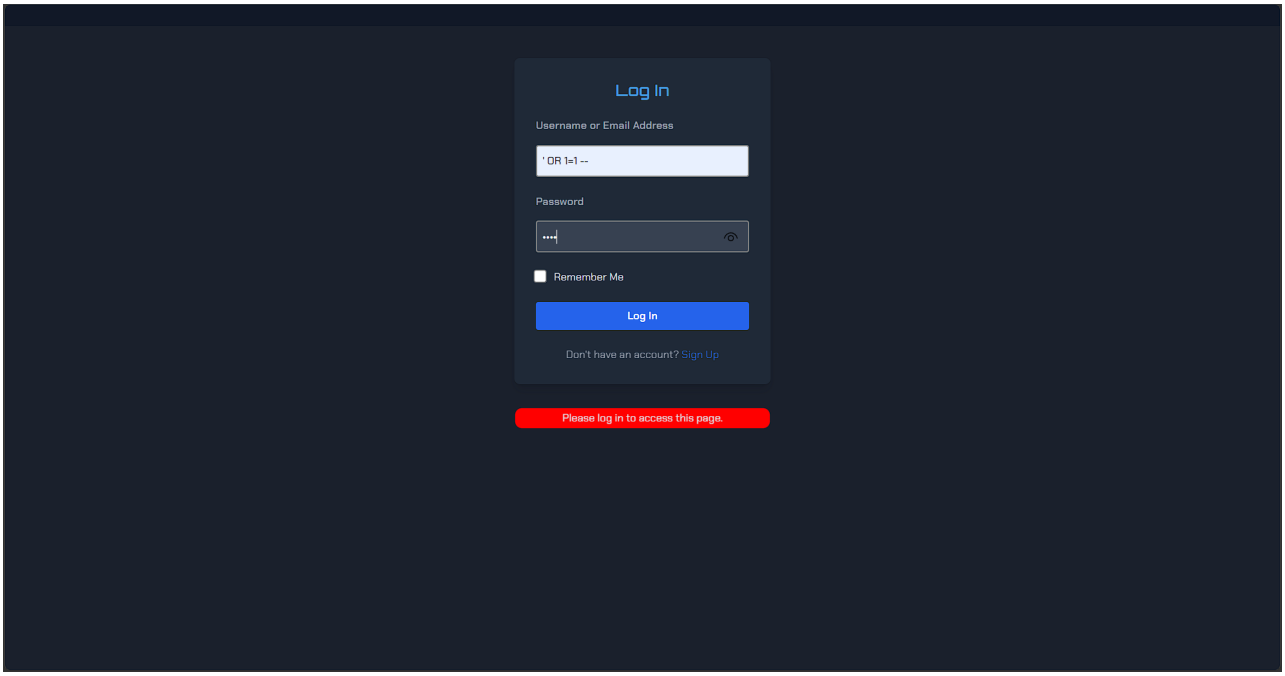
B17. Implement one of the current state-of-the-art solutions and evaluate it.

...

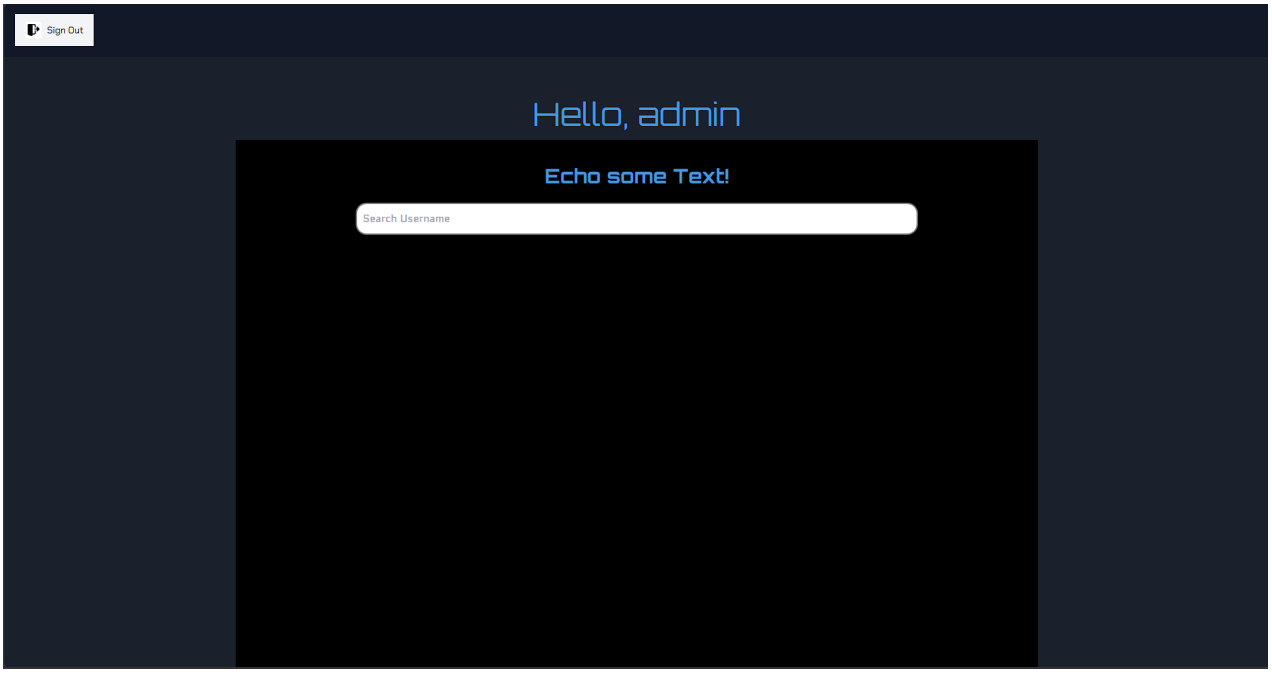
B21. Design and implement a cybersecurity learning activity.

For this activity, the learning activity I created was an activity to showcase SQL injection and command injection attacks. This was built as a Flask application, reusing some resources from my Agile Web Development project (as the application itself was not part of the activity).

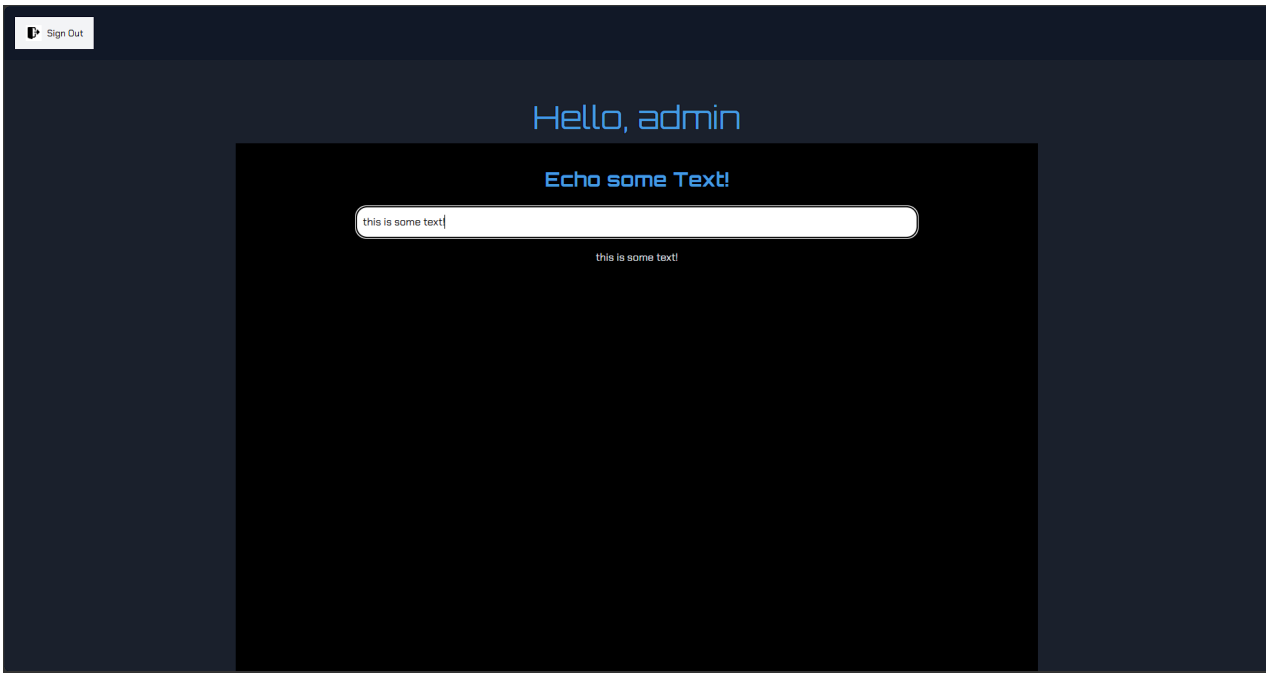
SQL injection is implemented using a poorly constructed SQL query interfaced with the `sqlite3` library. Against recommended advice, it uses a format string which makes it vulnerable to injection attacks.



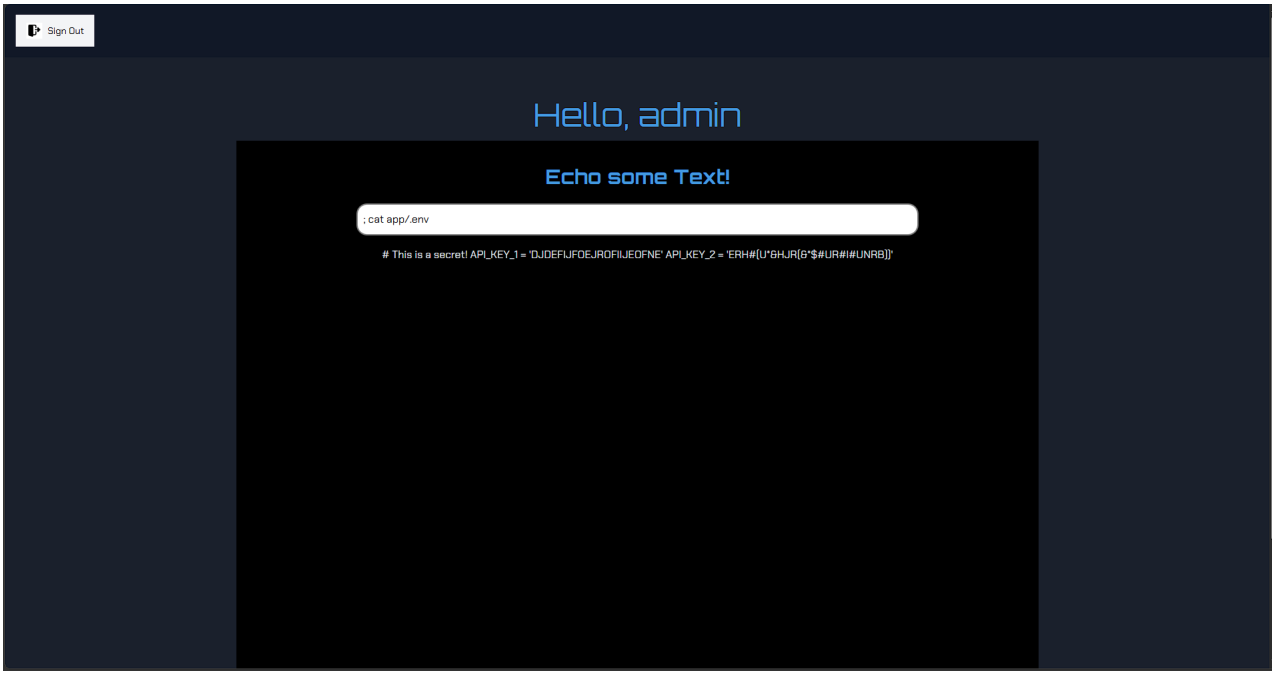
A dummy admin account is provided as the first user in the database, so an injected string will automatically be logged in as the admin user.



As the admin user, special privileges are given to them that no other user account has. On the homepage, they are able to enter a line of text and see it displayed on the page instantly.



However, this is implemented using the `os.system()` function. Consequently, it is vulnerable to command injection which can leak sensitive information.



B23. Test an intrusion detection system and discuss its effectiveness.

...

B24. Design and implement access control of your choice.

...

B25. Design and implement a threat intelligence module of your choice.

...

B28. Produce a cyber safety flyer for (choose 1): elders, high school students, CEOs, Uni students.

...

B29. Find a CVE in this year and fix it using three different generative AI systems (e.g., ChatGPT, Gemini), comparing the consistency.

To complete this activity, I began by going through a list of recently published CVEs. This was done using NIST's National Vulnerability Database, which provides a helpful search tool with filtering capabilities making it easy to locate recently published vulnerabilities. After some scrolling, I came across a few CVEs which looked interesting but I wanted to use one where the vulnerability had publicly accessible code that I could reference for the LLMs to actually fix since that would give the most varied response (at least in theory). Eventually I settled on CVE-2026-35363, which was published on April 22 2026 and impacted the `rm` implementation found in the uutils coreutils library.

The uutils coreutils library is a reimplementation of the GNU coreutils written in Rust, primarily being used to replace the C-based GNU coreutils in Linux-based operating systems. This vulnerability was found by Canonical, who audited the uutils coreutils implementations for security flaws before they could be used to replace the GNU coreutils currently used by Ubuntu. This audit led to 113 issues being identified, of which this was one of them. Although many of these issues were resolved before Ubuntu 26.04 LTS was released (and hence are currently being used as of writing this), the `cp`, `mv`, and `rm` commands are still using the GNU coreutils implementations with plans to replace them with the uutils coreutils implementations in Ubuntu 26.10.

The bug identified with CVE-2026-35363 is a lack of proper safeguard mechanisms for the uutils coreutils implementation of `rm`. Specificially, it is vulnerable to CWE-22 (improper limitation of a pathname to a restricted directory) making it vulnerable to path traversal attacks. The `rm` command should, normally, prevent the current directory (or the parent directory) from being removed. However, this implementation only prevent specific paths from being removed (i.e. `.` and `..`). It did not consider semantically equivalent paths, like `./` or `.././`, which could be used to delete all contents within the current directory. The reason this wasn't discovered earlier is because the command did report that it received an invalid input, but deleted everything in the specified directory regardless.

The three LLMs chosen were ChatGPT (using GPT-5.2), Claude (using Sonnet 4.6 Adaptive), and Gemini (using Gemini 3 Flash). In each case, the free tier was used (using the models available as of April 25 2026) to minimise the difference in model quality between vendors (i.e. it uses the "baseline" models for each to give a true representation of their capabilities) in addition to it being infeasible to pay for three different subscriptions to access higher tier models. Although this will give a baseline benchmark for LLMs, it should be noted that using higher tier models could give more effective output.

The following prompt was given to all models with no alterations.

```
Consider CVE-2026-35363. This is a vulnerability that affects the rm utility of
uutils coreutils, a Rust reimplementation of the GNU coreutils.

The following is a description of the issue:
A vulnerability in the rm utility of uutils coreutils allows the bypass of safeguard
mechanisms intended to protect the current directory. While the utility correctly
refuses to delete . or .., it fails to recognize equivalent paths with trailing slashes,
such as ./ or .///. An accidental or malicious execution of rm -rf ./ results in the
silent recursive deletion of all contents within the current directory. The command
further obscures the data loss by reporting a misleading 'Invalid input' error,
which may cause users to miss the critical window for data recovery.

This issue is identified by issue #9749 on the GitHub page for this library,
with the following description:
    Component
    rm -rf

    Description
    An issue with rm -rf: when deleting with rm -rf ., consistent with GNU rm, it
```

refuses to delete the current directory, reports an error directly, and does not delete any contents.

```
rm -rf .
rm: refusing to remove '.' or '..' directory: skipping '.'
```

However, when using rm -rf ./ or rm -rf .///, it reports an error as if the deletion failed (it should be consistent with rm -rf ., refusing directly without deleting anything), but in reality the contents of the current directory have already been deleted:

```
rm: cannot remove './': Invalid input
```

This is because the clean_trailing_slashes function collapses .// into "./":

```
fn clean_trailing_slashes(path: &Path) -> &Path {
    ...
        // Checks if element at the end is a '/'
        if path_bytes[idx] == dir_separator {
            for i in (1..path_bytes.len()).rev() {
                if path_bytes[i - 1] != dir_separator {
                    idx = i;
                    break;
                }
            }
            #[cfg(unix)]
            return Path::new(OsStr::from_bytes(&path_bytes[0..=idx]));
            ...
        }
    }
    path
}
```

However, the path_is_current_or_parent_directory function only matches ".", "..", etc. It does not match equivalent paths like "./" or "../".

```
/// Checks if the path is referring to current or parent directory, if it is
referring to current or any parent directory in the file tree e.g '/../..', '../..'
fn path_is_current_or_parent_directory(path: &Path) -> bool {
    let path_str = os_str_as_bytes(path.as_os_str());
    let dir_separator = MAIN_SEPARATOR as u8;
    if let Ok(path_bytes) = path_str {
        return path_bytes == ([b'.'])
            || path_bytes == ([b'.', b'.'])
            || path_bytes.ends_with(&[dir_separator, b'.'])
            || path_bytes.ends_with(&[dir_separator, b'.', b'.'])
            || path_bytes.ends_with(&[dir_separator, b'.', dir_separator])
            || path_bytes.ends_with(&[dir_separator, b'.', b'.', dir_separator]);
    }
    false
}
```

How would you fix this particular vulnerability, and why would you make that change?

Note: the code generated by each model will not be copied here. The chats have been provided for your own viewing, where the exact code generated can be seen. This will also not be tested because the codebase is too large to reasonably test the changes, and I don't know Rust well enough to be able to run the command properly. Instead, the conclusions each model comes to will be summarised and compared which should hopefully be enough.

ChatGPT (GPT-5.2)
ChatGPT identified that the cause of the bug was a lack of proper safety checks. To fix this bug, it recommended a few different options: - Normalising the path components before checking - Explicitly include semantically equivalent paths in `path_is_current_or_parent_directory`

The normalising approach uses the `Path` module in Rust's standard library, which has a `components` method which automatically normalises paths. This would allow it to handle all equivalent representations of the same directory, ensuring that this function doesn't silently fail.

The explicit path inclusion approach involves adding paths like `./` and `../` to the function, but it is noted that this is a weaker approach that will keep missing edge cases (as these paths can be arbitrarily long, such as `././[...]./`).

Claude (Sonnet 4.6 Adaptive)
Claude identified that the cause of the bug was `clean_trailing_slashes` strips trailing slahes but leaves one which is then used with `path_is_current_or_parent_directory`. It proposes a few different options to fix this bug: - Strip all trailing slahes in `clean_trailing_slashes` - Add the missing `./` and `../` patterns to `path_is_current_or_parent_directory`

The stripping trailing slashes approach involves removing all trailing slashes from the path string so that the path always ends in a non `/` character. This proposes this because it will normalise the provided paths, which then gets caught by `path_is_current_or_parent_directory`.

The adding missing patterns approach involves just adding 2 missing paths - `./` and `../`. It suggests this approach because the `path_is_current_or_parent_directory` should recognise any path that refers to the current or parent directory, even if the input isn't normalised.

It recommends using both of these approaches together because each approach misses something if it is implemented alone.

Gemini (Gemini 3 Flash)
Gemini identified the cause of the bug was the safeguard being too literal and the input path not being properly cleaned/normalised. The fix it proposes is to normalise the path or update the `path_is_current_or_parent_directory` function to be aware of the trailing `/` character. Gemini proposes this fix because it fixes the logical flaw and makes behaviour consistent with GNU coreutils.

Comparison
At the time of writing, no change has been made to this vulnerability yet. However, pull request #11005 has been created to address the problem, which proposes the following solution:

```
fn path_is_current_or_parent_directory(path: &Path) -> bool {
    let Ok(bytes) = os_str_as_bytes(path.as_os_str()) else {
        return false;
    };

    // Strip all trailing separators
    let trimmed = match bytes.iter().rposition(|&b| !is_separator(b)) {
        Some(i) => &bytes[..=i],
        None => return false, // all separators or empty
    };
```

```
        // Extract the last component (after the last separator)
        let last = match trimmed.iter().rposition(|&b| is_separator(b)) {
            Some(i) => &trimmed[i + 1..],
            None => trimmed,
        };

        last == b"." || last == b".."
    }
```

This is different to what each LLM proposed, and instead strips all the trailing `/` characters and checks that the last component is `.` or `...` Although this is effectively just normalising the path, the closest answer to this solution was provided by Claude.

The least surprising thing about the answers provided by each LLM is that the primary solution was to normalise the path, which the proposed solution also does. Additionally, each LLM makes reference to the potential data loss vulnerability created from the output generated when paths like `./` are passed. However, only ChatGPT and Claude provide an alternative solution. This alternative solution is the same between them, and involves explicitly adding missing paths to the final check. However, ChatGPT specifically makes note that this is a weak solution while Claude says that both solutions should be used together rather than choosing between one or the other.

An interesting thing that deviates between the models is their justification for their proposal. Both Claude and Gemini justify why they chose to implement their solution in the way they chose, which use reasonably sound logic. Gemini in particular gives several justifications for its reasoning, though every reason is semi-related (which can be forgiven). However, ChatGPT justifies why its implementation works. Although this is just a semantic difference since it ends up essentially being the same, its still an interesting shift regardless.

B30. Generate an AI-created image, applying an imperceptible watermark on it and then perform an image-to-image regeneration or editing process to make sure the watermark is detectable—the watermark survives.

...

References

National Institute of Standards and Technology. "NVD Vulnerability Search". Accessed: Apr. 24, 2026. [Online]. Available: https://nvd.nist.gov/vuln/search#/nvd/home?vulnRevisionStatusList=Published

National Institute of Standards and Technology. "CVE-2026-35363 Detail". Accessed: Apr. 25, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2026-35363

R. Sharma. "An update on rust-coreutils". Canonical Ltd. Accessed: Apr. 25, 2026. [Online]. Available: https://discourse.ubuntu.com/t/an-update-on-rust-coreutils/80773

The MITRE Corporation. "CVE-2026-35363". Accessed: Apr. 25, 2026. [Online]. Available: https://www.cve.org/CVERecc 2026-35363

The MITRE Corporation. "CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')". Accessed: Apr. 25, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/22.html

The Rust Team. "Module path". Accessed: Apr. 25, 2026. [Online]. Available: https://doc.rust-lang.org/std/path/index.html